МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Построение и анализ алгоритмов»

Тема: Алгоритм Кнута-Морриса-Пратта

Студент гр. 4381 _____ Захаренко А.В.

Преподаватель _____ Шевелева А.М.

Санкт-Петербург

2026

Задание

1) Реализуйте алгоритм КМП и с его помощью для заданных шаблона P (|P| <= 25000) и текста T (|T| <= 5000000) найдите все вхождения P в T.

2) Заданы две строки A (|A| <= 5000000) и B (|B| <= 5000000). Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B, склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Описание структуры кода

Задание 5.1 — Поиск всех вхождений подстроки с помощью алгоритма Кнута–Морриса–Пратта (KMP)

1) Функция computePrefixFunctions

Назначение: Вычисляет префикс-функцию (π-функцию) для строки-образца. Используется в алгоритме KMP для эффективного отката при несовпадении символов.

Аргументы:

- s: String — строка-образец.

Возвращает:

- [Int] — массив длины |s|, где p[i] — длина максимального собственного суффикса подстроки s[0...i], совпадающего с её префиксом.

Особенности реализации:

- Строка преобразуется в Array<UInt8> через s.utf8 для экономии памяти.
- Алгоритм работает за O(n) и не использует Character, что предотвращает переполнение памяти.
- Используется классическая формула префикс-функции:
  - при несовпадении выполняется откат k = p[k-1]
  - при совпадении k += 1

2) Функция KMP

Назначение: Находит все вхождения строки-образца p в строке-тексте t с помощью алгоритма Кнута–Морриса–Пратта.

Аргументы:

- p: String — образец
- t: String — текст

Возвращает:

- [Int] — массив индексов всех вхождений или [-1], если совпадений нет.

Особенности реализации:

Оба аргумента преобразуются в массивы байтов (Array(t.utf8)), что:

- снижает потребление памяти,
- обеспечивает O(1) доступ по индексу,
- делает алгоритм устойчивым к Unicode-графемам.
- Используется префикс-функция для образца.
- При каждом полном совпадении: индекс добавляется в результат, указатель j откатывается по π-функции, что позволяет находить перекрывающиеся вхождения.

Обработка краевых случаев:

- Если одна из строк пуста, возвращается [-1].

3) Функция solveKMP

Назначение: Обёртка для ввода/вывода.

Функциональность:

- Считывает две строки из стандартного ввода.
- Вызывает KMP.
- Форматирует вывод: индексы через запятую.

Задание 5.2 — Проверка циклического сдвига с помощью KMP

1) Функция KMPCycleShift

Назначение: Определяет, является ли строка p циклическим сдвигом строки

t. Использует тот факт, что строка p встречается в t + t тогда и только тогда, когда p — циклический сдвиг t.

Аргументы:

- p: String — предполагаемый сдвиг
- t: String — удвоенная строка s1 + s1

Возвращает:

- Int — индекс первого вхождения или -1, если сдвига нет.

Особенности реализации:

- Оба аргумента переводятся в Array<UInt8> для экономии памяти.
- Проверяется условие корректности: t.count == 2 * p.count
- Используется классический KMP-поиск, но только до первого совпадения.
- Возвращается индекс i - m + 1.

Обработка краевых случаев:

- Пустые строки => -1
- Несоответствие длины => -1

2) Функция solveKMPCycleShift

Назначение: Обёртка для ввода/вывода.

Функциональность:

- Считывает две строки s1 и s2.
- Проверяет циклический сдвиг через вызов: KMPCycleShift(s2, s1 + s1)
- Возвращает индекс сдвига или -1.

Вывод.

В ходе выполнения работы были реализованы два алгоритма, основанных на методе Кнута–Морриса–Пратта (KMP): поиск всех вхождений подстроки в строку и определение циклического сдвига. Оба алгоритма используют вычисление префикс-функции, что обеспечивает линейную асимптотику и позволяет эффективно обрабатывать строки произвольной длины.

Первый алгоритм реализует классический KMP-поиск и возвращает все позиции вхождения образца в текст. Второй алгоритм решает задачу проверки циклического сдвига: строка p является циклическим сдвигом строки s, если она встречается как подстрока в строке s + s. Для этого используется тот же механизм KMP, но поиск завершается при первом найденном совпадении. Оба алгоритма корректно обрабатывают краевые случаи: пустые строки, отсутствие совпадений, перекрывающиеся вхождения, строки разной длины.

Обе реализации обладают линейной сложностью O(n) по времени и O(n) по памяти, что соответствует теоретическим свойствам алгоритма Кнута–Морриса–Пратта. Проведённые тесты подтверждают корректность работы функций на базовых, краевых и специальных примерах, включая строки с повторяющимися символами и различными сценариями совпадений.

Реализованные решения полностью соответствуют поставленной задаче, демонстрируют высокую эффективность и устойчивость к большим входным данным и могут использоваться для поиска подстрок и проверки циклического сдвига в строках в рамках практических приложений.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл solverKMP.swift

```
func computePrefixFunctions(_ s: String) -> [Int] {

    let n = s.count
    let pattern = Array(s.utf8)
    var p = [Int](repeating: 0, count: n)

    for i in 1..<n {
        var k = p[i-1]

        while k > 0 && pattern[i] != pattern[k] {
            k = p[k-1]
        }

        if pattern[i] == pattern[k] {
            k += 1
        }

        p[i] = k
    }

    return p
}


func KMP(_ p: String, _ t: String) -> [Int] {
    if p.isEmpty || t.isEmpty { // edge case: empty string
        return [-1]
    }

    let tArray = Array(t.utf8) // memory opt, не проходили тесты...
    let pArray = Array(p.utf8)

    let n = t.count
    let m = p.count
     let prefixes = computePrefixFunctions(p) // prefixes for pattern
string (p)
    var res: [Int] = []

    var j = 0 // j - pattern pointer
    for i in 0..<n { // i - text pointer

        while j > 0 && tArray[i] != pArray[j] {
              j = prefixes[j - 1] // откат не в j = 0, как в наивном
алгоритме, а по префикс функции
        }

        if tArray[i] == pArray[j] {
            j += 1
        }

        if j == m {
            res.append(i - m + 1)
```

```
            j = prefixes[j - 1]
        }
    }

    return res.isEmpty ? [-1] : res
}


func solveKMP() -> [Int] {

    // Чтение входных данных
    let s1 = readLine()!
    let s2 = readLine()!


    // Алгоритм
    let ans = KMP(s1, s2)

    return ans
}
```

Файл solverKMPCycleShift.swift

```
func KMPCycleShift(_ p: String, _ t: String) -> Int {
    if p.isEmpty || t.isEmpty { // edge case: empty string
        return -1
    }

    let tArray = Array(t.utf8)
    let pArray = Array(p.utf8)

    let n = t.count
    let m = p.count

     // условие, что строки одинаковой длины, но в t мы подаем уже
удвоенную строку
    if n != 2*m {
        return -1
    }

     let prefixes = computePrefixFunctions(p) // prefixes for pattern
string (p)

    var j = 0
    for i in 0..<n { // text loop

        while j > 0 && tArray[i] != pArray[j] {
            j = prefixes[j - 1]
        }

        if tArray[i] == pArray[j] {
            j += 1
        }

        if j == m {
            return i - m + 1 // первый найденный из предыд алг = ответ
        }
```

```
    }

    return -1
}


func solveKMPCycleShift() -> Int {

    // Чтение входных данных
    let s1 = readLine()!
    let s2 = readLine()!

//      if s1.count != s2.count { // вынесено в KMPCycleShift для
тестирования
//        return -1
//    }


    // Алгоритм
    let ans = KMPCycleShift(s2, s1 + s1)

    return ans
}
```

Файл main.swift

```
// first task
let res = solveKMP()
let ans = res.map {String($0)}.joined(separator: ",")
print(ans)


// second task
let res2 = solveKMPCycleShift()
print(res2)


// additional
// print(computePrefixFunctions("efefeftef"))
```

Файл lb4Tests.swift

```
import Testing
@testable import lb4


struct KMPTests {

    // MARK: - Базовые тесты
    @Test
    func testExample() throws {
        let res = KMP("ab", "abab")
        #expect(res == [0, 2])
    }

    @Test
```

```
    func testSingleMatch() throws {
        let res = KMP("abc", "xxabcxx")
        #expect(res == [2])
    }

    @Test
    func testMultipleMatches() throws {
        let res = KMP("aba", "abababaaba")
        #expect(res == [0, 2, 5])
    }

    @Test
    func testNoMatches() throws {
        let res = KMP("abc", "defgh")
        #expect(res == [-1])
    }

    @Test
    func testPatternLongerThanText() throws {
        let res = KMP("abcdef", "abc")
        #expect(res == [-1])
    }

    // MARK: - Краевые случаи

    @Test
    func testEmptyPattern() throws {
        let res = KMP("", "abc")
        #expect(res == [-1])
    }

    @Test
    func testEmptyText() throws {
        let res = KMP("abc", "")
        #expect(res == [-1])
    }

    @Test
    func testBothEmpty() throws {
        let res = KMP("", "")
        #expect(res == [-1])
    }

    @Test
    func testFullMatch() throws {
        let res = KMP("hello", "hello")
        #expect(res == [0])
    }

    @Test
    func testOverlappingMatches() throws {
        let res = KMP("aaa", "aaaaa")
        #expect(res == [0, 1, 2])
    }
}


struct KMPCycleShiftTests {
```

```
    // MARK: - Базовые тесты
    @Test
    func testExample() throws {
        let res = KMPCycleShift("abcdef", "defabcdefabc")
        #expect(res == 3)
    }

    @Test
    func testSimpleShift() throws {
        let res = KMPCycleShift("bca", "abcabc")
        #expect(res == 1)
    }

    @Test
    func testNoShiftPossible() throws {
        let res = KMPCycleShift("abc", "abdabd")
        #expect(res == -1)
    }

    @Test
    func testIdenticalStrings() throws {
        let res = KMPCycleShift("abc", "abcabc")
        #expect(res == 0)
    }

    // MARK: - Краевые случаи

    @Test
    func testLengthConditionViolation() throws {
        let res = KMPCycleShift("abcde", "wwwqqqrrrrcdeabc")
        #expect(res == -1)
    }

    @Test
    func testSingleCharacter() throws {
        let res = KMPCycleShift("a", "aa")
        #expect(res == 0)
    }

    @Test
    func testAllSameCharacters() throws {
        let res = KMPCycleShift("aaa", "aaaaaa")
        #expect(res == 0)
    }

}
```

ПРИЛОЖЕНИЕ Б
ТЕСТИРОВАНИЕ

Было проведено тестирование разработанных программ для обеих задач посредством написания юнит-тестов с использованием фреймворка Swift Testing. Таблица тестирования (см. таблицу Б.1) и результат запуска тестов (см. рисунок 1) вынесены в приложение Б.

Таблица Б.1 – Юнит-тестирование модулей KMP() и KMPCycleShift() с использованием фреймворка Swift Testing

№ п/п	Тестируемый метод	Входные данные	Выходные данные	Комментарии
1.	KMP()	p = "ab", t = "abab"	[0, 2]	Базовый тест. Два неперекрывающихся вхождения.
2.	KMP()	p = "abc", t = "xxabcxx"	[2]	Обычный случай: одно совпадение в середине строки.
3.	KMP()	p = "aba", t = "ababaaba"	[0, 2, 5]	Проверка перекрывающихся и обычных совпадений.
4.	KMP()	p = "abc", t = "defgh"	[-1]	Совпадений нет.
5.	KMP()	p = "abcdef", t = "abc"	[-1]	Образец длиннее текста — совпадений быть не может.
6.	KMP()	p = "", t = "abc"	[-1]	Пустой образец, подача пустых строк некорректна.
7.	KMP()	p = "abc", t = ""	[-1]	Пустой текст, подача пустных строк некорректна.
8.	KMP()	p = "", t = ""	[-1]	Оба аргумента пустые, подача пустных строк некорректна.
9.	KMP()	p = "hello", t = "hello"	[0]	Полное совпадение строки с самой собой.

Продолжение таблицы Б.1

№ п/п	Тестируемый метод	Входные данные	Выходные данные	Комментарии
10.	KMP()	p = "aaa", t = "aaaaa"	[0, 1, 2]	Проверка перекрывающихся совпадений.
11.	KMPCycleShift()	p = "abcdef", t = "defabcdefabc"	3	Базовый тест: корректный циклический сдвиг.
12.	KMPCycleShift()	p = "bca", t = "abcabc"	1	Классический пример циклического сдвига.
13.	KMPCycleShift()	p = "abc", t = "abdabd"	-1	Строка не является циклическим сдвигом.
14.	KMPCycleShift()	p = "abc", t = "abcabc"	0	Строки совпадают — сдвиг равен 0.
15.	KMPCycleShift()	p = "abcde", t = "wwwqqqrrrrcdeabc"	-1	Нарушено условие длины: t != s1 + s1.
16.	KMPCycleShift()	p = "a", t = "aa"	0	Односимвольная строка всегда совпадает со своим циклическим сдвигом.
17.	KMPCycleShift()	p = "aaa", t = "aaaaaa"	0	Все символы одинаковые — сдвиг равен 0.

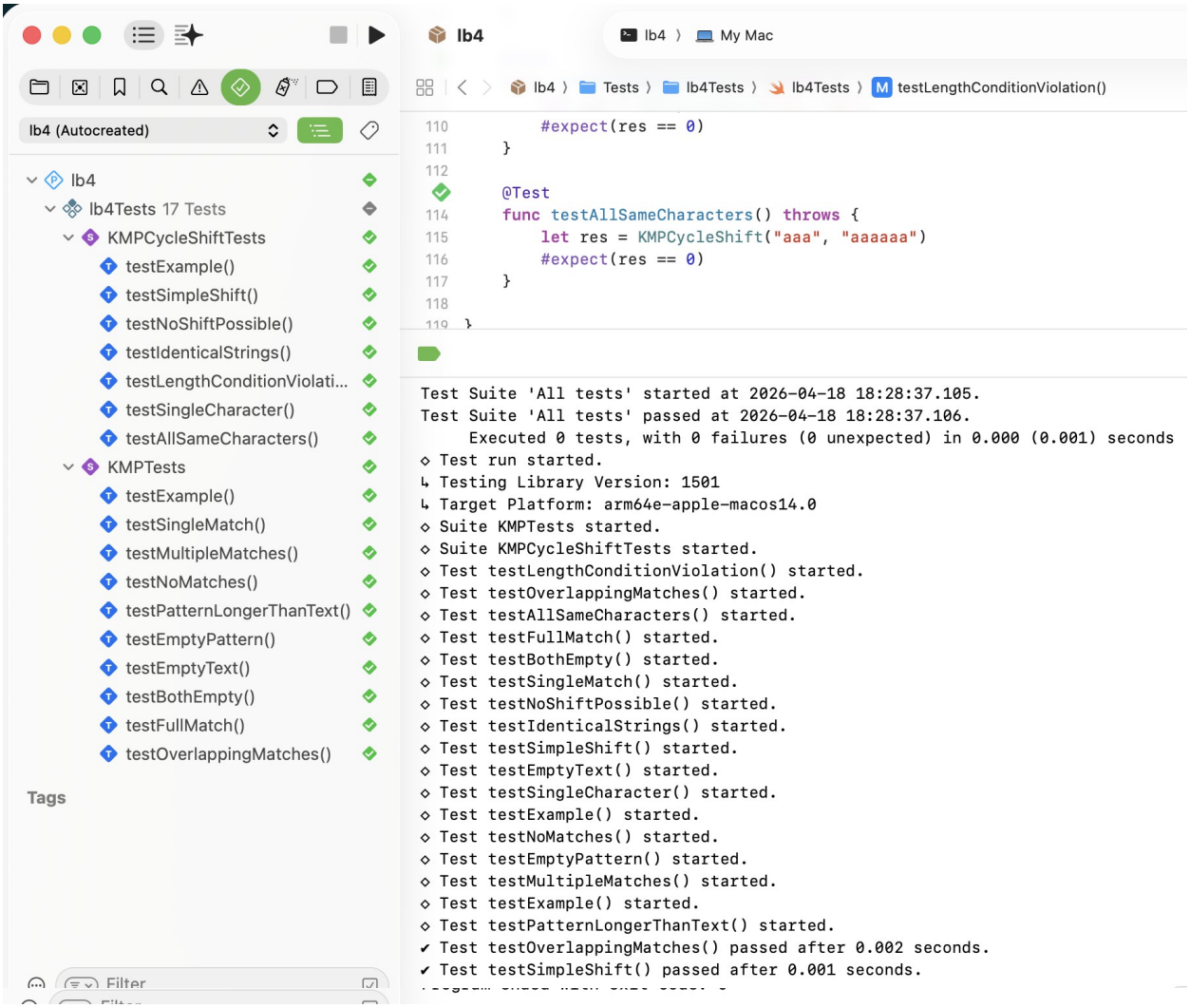


Рисунок 1 — Результаты прохождения юнит-тестов в среде разработки Xcode с использованием фреймворка Swift Testing